

Premiers pas avec SQL

1. Comment créer des tables ?

On reprend ici la base de données relationnelles vue en cours :

Relation *livre* :

id	titre	id_auteur	ann_publi	note
1	1984	1	1949	10
2	Dune	2	1965	8
3	Fondation	3	1951	9
4	Fahrenheit 451	4	1953	7
5	Ubik	5	1969	9
6	Chroniques martiennes	4	1950	8
7	La nuit des temps	6	1968	7
8	Blade Runner	5	1968	8
9	Les Robots	3	1950	9
10	La planète des singes	7	1963	8
11	Ravage	6	1943	8
12	Le Maître du Haut Château	5	1962	8
13	La fin de l'éternité	3	1955	8
14	De la Terre à la Lune	8	1865	10

Relation *auteur* :

id	nom	prenom	ann_naiss	langue_ecrit
1	Orwell	Georges	1903	anglais
2	Herbert	Frank	1920	anglais
3	Asimov	Isaac	1920	anglais
4	Bradbury	Ray	1920	anglais
5	K. Dick	Philip	1928	anglais
6	Barjavel	René	1911	français
7	Boulle	Pierre	1912	français
8	Verne	Jules	1828	français

Pour créer la 1ère table, on écrit ceci en langage SQL (pensez à exécuter les cellules à chaque fois) :

```
Entrée[7]: /* création de la table livre*/
DROP TABLE IF EXISTS livre;           -- au cas où une table livre ex
CREATE TABLE livre (                  -- DROP TABLE livre supprime la
    id INTEGER PRIMARY KEY NOT NULL,    -- on définit l'attribut id
    titre TEXT NOT NULL,                -- le NOT NULL signifie qu'
    id_auteur INTEGER REFERENCES auteur (id) NOT NULL, -- on définit
    ann_publi INTEGER,
    note INTEGER,
    CHECK (note>=0 AND note<=10)      -- pour spécifier que la no
);
```

Remarque.

Le langage SQL ne tient pas du tout compte de la casse (autrement dit, pour lui minuscule=minuscule). Pour plus de lisibilité, on mettra les mots clés en majuscule et le reste en minuscule.

Exercice 1

Créer la table *auteur* en langage SQL puis exécuter la cellule (une seule fois) :

```
Entrée[8]: /* création de la table auteur*/
-- à vous d'écrire le code en langage SQL en vous aidant de la créatic
DROP TABLE IF EXISTS auteur ;
CREATE TABLE auteur (
    id INTEGER PRIMARY KEY NOT NULL,
    nom TEXT NOT NULL,
    prenom TEXT NOT NULL,
    ann_naiss INTEGER NOT NULL,
    langue_ecrit TEXT NOT NULL
);
```

Remarque.

En principe, on aurait dû créer la table *auteur* avant la table *livre* car un attribut de la table *livre* (*id_auteur*) fait référence à un attribut de la table *auteur* (*id*). Pour des raisons pédagogiques, j'ai fait le choix de créer la table la plus complexe avant, à savoir la table *livre*.

2. Comment insérer des enregistrements ?

```
Entrée[9]: /* remplissage de la table auteur */
INSERT INTO auteur
VALUES
(1, 'Orwell', 'Georges', 1903, 'anglais'),
(2, 'Herbert', 'Frank', 1920, 'anglais'),
(3, 'Asimov', 'Isaac', 1920, 'anglais'),
(4, 'Bradbury', 'Ray', 1920, 'anglais'),
(5, 'K. Dick', 'Philip', 1928, 'anglais'),
(6, 'Barjavel', 'René', 1911, 'français'),
(7, 'Boulle', 'Pierre', 1912, 'français'),
(8, 'Verne', 'Jules', 1828, 'français');
```

```
Entrée[10]: /* remplissage de la table livre */
INSERT INTO livre
VALUES
(1, '1984', 1, 1949, 10),
(2, 'Dune', 2, 1965, 8),
(3, 'Fondation', 3, 1951, 9),
(4, 'Fahrenheit 451', 4, 1953, 7),
(5, 'Ubik', 5, 1969, 9),
(6, 'Chroniques martiennes', 4, 1950, 8),
(7, 'La nuit des temps', 6, 1968, 7),
(8, 'Blade Runner', 5, 1968, 8),
(9, 'Les Robots', 3, 1950, 9),
(10, 'La planète des singes', 7, 1963, 8),
(11, 'Ravage', 6, 1943, 8),
(12, 'Le Maître du Haut Château', 5, 1962, 8),
(13, 'La fin de l''éternité', 3, 1955, 8),
(14, 'De la Terre à la Lune', 8, 1865, 10);
```

L'intérêt d'une contrainte d'unicité

Exécutez la cellule suivante et constatez qu'elle produit une erreur. On ne peut pas en effet avoir de doublons pour une clef primaire !

```
Entrée[11]: INSERT INTO auteur
VALUES
(1, 'Daudet', 'Alphonse', 1840, 'français');
```

Error: UNIQUE constraint failed: auteur.id

Par contre, l'insertion suivante est valide (exécutez-là mais une seule fois) :

```
Entrée[12]: INSERT INTO auteur
VALUES
(9, 'Daudet', 'Alphonse', 1840, 'français');
```

L'intérêt d'une contrainte métier

Exécutez la cellule suivante et constatez qu'elle produit également une erreur. En effet, quand on a défini la table, on a spécifié qu'une note devait toujours être comprise entre 0 et 10.

```
Entrée[13]: INSERT INTO livre  
VALUES  
(15, 'Les lettres de mon moulin',9,1869,11);
```

Error: CHECK constraint failed: note>=0 AND note<=10

Par contre ça fonctionne très bien quand on respecte les différentes contraintes. Exécutez la cellule suivante et constatez qu'il n'y a pas d'erreur.

```
Entrée[15]: INSERT INTO livre  
VALUES  
(15, 'Les lettres de mon moulin',9,1869,6);
```

Error: UNIQUE constraint failed: livre.id

3. Comment écrire des requêtes ?

Pour visualiser une table entière : `SELECT * FROM <nom_table>`

Exécutez la cellule suivante et constatez qu'on visualise tous les attributs de la table livre.

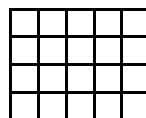
Entrée[16]: **SELECT * FROM livre;**

Sortie[16]:

id	titre	id_auteur	ann_publi	note
1	1984	1	1949	10
2	Dune	2	1965	8
3	Fondation	3	1951	9
4	Fahrenheit 451	4	1953	7
5	Ubik	5	1969	9
6	Chroniques martiennes	4	1950	8
7	La nuit des temps	6	1968	7
8	Blade Runner	5	1968	8
9	Les Robots	3	1950	9
10	La planète des singes	7	1963	8
11	Ravage	6	1943	8
12	Le Maître du Haut Château	5	1962	8
13	La fin de l'éternité	3	1955	8
14	De la Terre à la Lune	8	1865	10
15	Les lettres de mon moulin	9	1869	6

**La projection : SELECT <nom_attribut>, <nom_attribut>
FROM <nom_table>**

Pour sélectionner une ou plusieurs colonnes :



Exécutez la cellule suivante et constatez qu'on visualise seulement les attributs titre, note et ann_publ (dans cet ordre) de la table livre

Entrée[17]: **SELECT** titre,note,ann_publi **FROM** livre;

Sortie[17]:

	titre	note	ann_publi
	1984	10	1949
	Dune	8	1965
	Fondation	9	1951
	Fahrenheit 451	7	1953
	Ubik	9	1969
	Chroniques martiennes	8	1950
	La nuit des temps	7	1968
	Blade Runner	8	1968
	Les Robots	9	1950
	La planète des singes	8	1963
	Ravage	8	1943
	Le Maître du Haut Château	8	1962
	La fin de l'éternité	8	1955
	De la Terre à la Lune	10	1865
	Les lettres de mon moulin	6	1869

avec le mot clé AS, il est possible de renommer des colonnes ou même de créer de nouvelles

Entrée[18]: **SELECT titre,ann_publi AS année_de_publication,note AS note_sur_10 FROM**

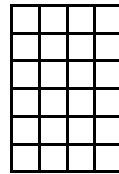
Sortie[18]:

titre	année_de_publication	note_sur_10
1984	1949	10
Dune	1965	8
Fondation	1951	9
Fahrenheit 451	1953	7
Ubik	1969	9
Chroniques martiennes	1950	8
La nuit des temps	1968	7
Blade Runner	1968	8
Les Robots	1950	9
La planète des singes	1963	8
Ravage	1943	8
Le Maître du Haut Château	1962	8
La fin de l'éternité	1955	8
De la Terre à la Lune	1865	10
Les lettres de mon moulin	1869	6

Entrée[]: **SELECT titre,ann_publi AS année_de_publication,note*2 AS note_sur_20 FROM**

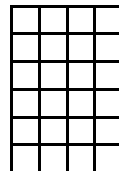
Restriction : **SELECT * FROM <nom_table> WHERE <condition>**

Pour sélectionner toutes les lignes vérifiant la condition donnée



restriction avec projection : **SELECT * FROM <nom_table> WHERE <condition>**

On va combiner les deux pour obtenir seulement les cases rouges



les conditions courantes :

`<nom_attribut> [=, >, <, <=, >=] <valeur>`

`<nom_attribut> LIKE <chaîne de caractères avec des caractères génériques comme % ou _>`

Le caractère `_` représente n'importe quel caractère et `%` n'importe quelle chaîne de caractères. On peut les insérer n'importe où dans la chaîne de caractères de comparaison.

Par ex :

```
SELECT * FROM auteur WHERE nom LIKE '%a%'`
```

sélectionne toutes les lignes dont le `*nom*` est une chaîne de caractères contenant un `*a*` (ou un `*A*` : le langage SQL ne tient pas compte de la casse).

Exécutez les 3 cellules suivantes et vérifiez à chaque fois qu'on visualise les lignes vérifiant la condition donnée

Entrée[19]: **SELECT titre,note FROM livre WHERE note>=9;**

Sortie[19]:

	titre	note
	1984	10
	Fondation	9
	Ubik	9
	Les Robots	9
	De la Terre à la Lune	10

Entrée[32]: **SELECT nom,prenom,ann_naiss AS année_de_naissance FROM auteur WHERE n**

Sortie[32]:

	nom	prenom	année_de_naissance
	Asimov	Isaac	1920
	Bradbury	Ray	1920
	Barjavel	René	1911
	Daudet	Alphonse	1840

Entrée[]: **SELECT nom,prenom,ann_naiss AS année_de_naissance FROM auteur WHERE no**

Entrée[]: **SELECT titre,ann_publi AS année_de_publication FROM livre WHERE ann_pu**

Voici deux instructions (à exécuter) permettant d'afficher tous les titres et leurs années de publication dont l'année de publication est comprise (au sens large) entre 1962 et 1969.

Entrée[]: **SELECT titre,ann_publi AS année_de_publication FROM livre WHERE ann_pu**

Entrée[]: **SELECT titre,ann_publi AS année_de_publication FROM livre WHERE ann_pu**

On peut aussi vouloir afficher tous les films et leurs notes dont l'année de publication est compris entre 1962 et 1969.

Entrée[]: **SELECT titre,note FROM livre WHERE ann_publi>=1962 AND ann_publi<=1969**

Exercice 2

Question 1

Afficher tous les titres commençant par la lettre L

Entrée[]: `-- rentrer votre requête SQL ici --`

Question 2

Afficher les noms, prénoms et année de naissance de tous les auteurs dont l'année de naissance est strictement inférieur à 1900 ou strictement supérieur à 1920.

Entrée[]: `-- rentrer votre requête SQL ici --`

Le tri avec la clause ORDER BY en fin de requête

Pour afficher tous les films par ordre alphabétique avec leurs années de publication :

Entrée[]: `SELECT titre,ann_publi AS année_de_publication FROM livre ORDER BY tit`

Comme par défaut, le tri est croissant, on peut omettre ASC (pour ascendant) :

Entrée[]: `SELECT titre,ann_publi AS année_de_publication FROM livre ORDER BY tit`

Pour trier par ordre décroissant, on précise le mot clé DESC (pour descendant) :

Entrée[]: `SELECT titre,ann_publi AS année_de_publication FROM livre ORDER BY tit`

Suppression des doublons avec le mot clé DISTINCT

Pour afficher les différentes langues des auteurs sans les doublons :

Entrée[]: `SELECT DISTINCT langue_ecrit FROM auteur;`

Exercice 3

Question 1

Afficher tous les titres par ordre croissant d'année de publication

Entrée[]: `-- rentrer votre requête SQL ici --`

Question 2

Afficher toutes les années de naissance des auteurs sans les doublons dans l'ordre décroissant

Entrée[]: `-- rentrer votre requête SQL ici --`

Fonctions d'agrégation

Les fonctions d'agrégation permettent d'appliquer une fonction à l'ensemble des valeurs d'une colonne (ou de plusieurs colonnes) et de renvoyer le résultat comme une table à 1 seule ligne et 1 seule colonne.

Voici quelques fonctions d'agrégations :

- **COUNT** permettant de compter le nombre d'entrées non vide
- **AVG** (pour *average* en anglais qui signifie moyenne) permettant de calculer la moyenne d'une colonne
- **SUM** permettant de calculer la somme d'une colonne
- **MIN** et **MAX** permettant de trouver respectivement le minimum et le maximum d'une colonne

Dans une telle requête, il faut bien décomposer les choses pour construire la requête : d'abord, le résultat qui va faire un certain nombre de lignes, puis la fonction d'agrégation qui est appliquée sur ce résultat.

Exécutez la cellule suivante et vérifiez qu'elle affiche bien le nombre de lignes de la table livre

Entrée[]: `SELECT COUNT (*) FROM livre;`

On préfère renommer le titre avec le mot clé **AS**

Entrée[]: `SELECT COUNT (*) AS Total FROM livre;`

On peut vouloir obtenir la note moyenne des livres :

Entrée[]: `SELECT AVG (note) AS note_moyenne FROM livre;`

ou bien obtenir le nombre d'auteurs anglais dans la table auteur

Entrée[]: `SELECT COUNT (*) AS nombre_d_auteurs_anglais FROM auteur WHERE langue_`

ou bien connaître le nombre de langues distinctes de la table auteur

Entrée[]: **SELECT COUNT (DISTINCT langue_ecrit) AS nombre_de_langues FROM auteur;**

Exercice 4

Question 1

Afficher le nombre de livres dont l'année de publication est comprise (au sens large) entre 1940 et 1960

Entrée[]: *-- rentrer votre requête SQL ici --*

Question 2

Afficher l'année de publication la plus récente dont les titres commencent par la lettre L

Entrée[34]: **SELECT MAX(ann_publi)
FROM livre
WHERE titre like 'L%'**

Sortie[34]:

1
1
1
1
1
1
1
1

Jointures

Il arrive que l'information sur une entité soit dispersée dans deux tables. On doit alors utiliser une jointure.

Par ex. quels sont les titres des livres écrits par K. Dick ?

Quand l'information est dispersée dans deux tables T1 et T2, la jointure consiste à construire d'abord une sorte de grosse table où chaque ligne est composée d'une ligne de T1 et d'une ligne de T2 (on appelle cette étape le produit cartésien de T1 avec T2). Cette grosse table contient le nombre de lignes dans T1 multiplié par le nombre de lignes dans T2.

Entrée[]: **SELECT * FROM livre JOIN auteur;**

Le produit cartésien contient encore beaucoup trop d'informations et une telle jointure n'a pas grand intérêt.

En face de chaque ligne d'un livre, on veut seulement l'auteur correspondant (et non pas comme précédemment tous les auteurs).

En effet, les colonnes *id_auteur* de la table *livre* et *id* de la table *auteur* ont les mêmes valeurs. On peut alors faire une jointure sur les valeurs des ces colonnes similaires, qui vont servir de point de branchement.

Entrée[20]: **SELECT * FROM livre JOIN auteur ON livre.id_auteur=auteur.id; -- on p**

Sortie[20]:

	id	titre	id_auteur	ann_publi	note	id	nom	prenom	ann_naiss	langue_ecri
	1	1984	1	1949	10	1	Orwell	Georges	1903	anglai
	2	Dune	2	1965	8	2	Herbert	Frank	1920	anglai
	3	Fondation	3	1951	9	3	Asimov	Isaac	1920	anglai
	4	Fahrenheit 451	4	1953	7	4	Bradbury	Ray	1920	anglai
	5	Ubik	5	1969	9	5	K. Dick	Philip	1928	anglai
	6	Chroniques martiennes	4	1950	8	4	Bradbury	Ray	1920	anglai
	7	La nuit des temps	6	1968	7	6	Barjavel	René	1911	françai
	8	Blade Runner	5	1968	8	5	K. Dick	Philip	1928	anglai

C'est déjà beaucoup plus satisfaisant. Mais on peut vouloir ne pas afficher les colonnes avec les identifiants

Entrée[]: **SELECT livre.titre,livre.ann_publi,livre.note,auteur.nom,auteur.prenom
FROM livre
JOIN auteur ON livre.id_auteur=auteur.id;**

On peut aussi mettre des alias avec le mot clé **AS** pour raccourcir l'instruction

Entrée[21]: **SELECT** l.titre,l.ann_publi,l.note,a.nom,a.prenom,a.ann_naiss,a.langue
FROM livre **AS** l
JOIN auteur **AS** a **ON** l.id_auteur=a.id;

Sortie[21]:

	titre	ann_publi	note	nom	prenom	ann_naiss	langue_ecrit
	1984	1949	10	Orwell	Georges	1903	anglais
	Dune	1965	8	Herbert	Frank	1920	anglais
	Fondation	1951	9	Asimov	Isaac	1920	anglais
	Fahrenheit 451	1953	7	Bradbury	Ray	1920	anglais
	Ubik	1969	9	K. Dick	Philip	1928	anglais
	Chroniques martiennes	1950	8	Bradbury	Ray	1920	anglais
	La nuit des temps	1968	7	Barjavel	René	1911	français
	Blade Runner	1968	8	K. Dick	Philip	1928	anglais
	Les Robots	1950	9	Asimov	Isaac	1920	anglais
	La planète des singes	1963	8	Boulle	Pierre	1912	français
	Ravage	1943	8	Barjavel	René	1911	français
	Le Maître du Haut Château	1962	8	K. Dick	Philip	1928	anglais
	La fin de l'éternité	1955	8	Asimov	Isaac	1920	anglais
	De la Terre à la Lune	1865	10	Verne	Jules	1828	français
	Les lettres de mon moulin	1869	6	Daudet	Alphonse	1840	français

On peut vouloir afficher tous les livres écrits par K.Dick

Entrée[]: **SELECT** l.titre,l.ann_publi,l.note
FROM livre **AS** l
JOIN auteur **AS** a **ON** l.id_auteur=a.id
WHERE a.nom='K. Dick';

Remarque. Il est possible de faire des jointures sur plus de 2 tables, il suffit de rajouter des JOIN <table> ON <condition de jointure> .

On peut vouloir calculer la moyenne des notes des livres de K. Dick

Entrée[]: **SELECT** AVG (l.note) **AS** note_moyenne
FROM livre **AS** l
JOIN auteur **AS** a **ON** l.id_auteur=a.id
WHERE a.nom='K. Dick';

Exercice 5

Question 1

Afficher tous les titres des livres écrits en français

Entrée[]: `-- rentrer votre requête SQL ici --`

Question 2

Afficher le nombre de livres écrits en français

Entrée[]: `-- rentrer votre requête SQL ici --`

Question 3

Afficher les titres des livres, les noms et prénoms des auteurs ainsi que l'âge qu'ils avaient lors de la publication (dans l'ordre décroissant de l'âge)

Entrée[]: `-- rentrer votre requête SQL ici --`

Question 4

Afficher l'âge moyen publication

Entrée[]: `-- rentrer votre requête SQL ici --`

4. Comment modifier des données ?

Les données stockées dans une SGBD ne sont *a priori* pas figées et peuvent être modifiées au cours du temps. Nous allons montrer deux types de modifications pouvant être faites sur les tables :

- la suppression d'un ensemble de lignes
- la mise à jour de certains attributs d'un ensemble de lignes

La suppression des lignes avec **DELETE FROM <table> WHERE <condition>**

Permet de supprimer toutes les lignes de la table vérifiant la condition donnée.

```
Entrée[ ]: DELETE FROM livre WHERE ann_publi>1950;
```

Vérifions que l'instruction précédente a bien supprimé de la table livre toutes les lignes dont l'année de publication est strictement supérieure à 1950 :

```
Entrée[ ]: SELECT * FROM livre;
```

Attention à ne pas confondre les instructions **DELETE FROM <table>** et **DROP TABLE <table>**

- La 1ère supprime toutes les lignes de la table mais la table existe et on peut donc insérer des éléments,
- La 2nde supprime totalement la table.

La mise à jour avec **UPDATE <table> SET <attribut>=<nouvelle_valeur> WHERE <condition>**

Sélectionne toutes les lignes de la table vérifiant la condition donnée et, pour chacune de ces lignes, remplace la valeur courante de l'attribut donné par la nouvelle valeur.

```
Entrée[ ]: UPDATE auteur SET langue_ecrit='english' WHERE langue_ecrit='anglais';
```

Vérifiez que la table *auteur* a été correctement mise à jour en exécutant la cellule suivante

```
Entrée[ ]: SELECT * FROM auteur;
```

Exercice 6

Question 1

Supprimer de la table **auteur** tous ceux nés au XIXe siècle (on rappelle à cette occasion que le XIXe siècle commence le 1er janvier 1801 et finit le 31 décembre 1900).

```
Entrée[ ]: -- rentrer votre requête SQL ici --
```

Vérifiez que la table *auteur* a été correctement mise à jour en exécutant la cellule suivante

```
Entrée[ ]: SELECT * FROM auteur;
```

Question 2

Les éditeurs se sont aperçus qu'ils ont noté trop large. Ils décident d'enlever 3 pts à chaque note. Faites cette modification.

Entrée[]: `-- rentrer votre requête SQL ici --`

Vérifiez que la table *livre* a été correctement mise à jour en exécutant la cellule suivante

Entrée[]: `SELECT * FROM livre;`